CSE 150: Introduction to Computer Networks

Chris Parsa

TCP Congestion Control

1

Reading Assignment and Problems

- Chapter 3: 3.5, 3.6, 3.7
  - Section 3.4.2: Pipelined Reliable Data Transfer Protocols
  - Section 3.5: Connection-oriented transport (TCP)
  - Section 3.6: Principles of congestion control
  - Section 3.7: TCP Congestion Control
  - End of Chapter problems for Congestion Control and TCP:
    - sections 3.5 - 3.7 (R14, R15, R16, R17, R18 and R19)
    - Extra problems at end of chapter as needed for practice
    - Make a study group – practice with friends!
  - Posted problems for Ch 3
- IMPORTANT: Section 2.7 on sockets is helpful for the Project

2

Project

- Sign up for a project group
- This week in lab:
  - State diagram
  - Project quiz
- More details in lab next week
- IMPORTANT: Section 2.7 on sockets is helpful for the Project

3

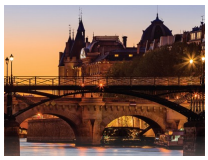
Upcoming

- Lab 6 part 1 demo
  - Demos ONLY in your registered section
  - Did you submit your code?
- Lab 6 part 2
- Test 2: Tuesday
  - Covers material since Test 1
- Group Assignment 5

5

Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
  - segment structure
  - reliable data transfer
  - flow control
  - connection management
- Principles of congestion control
- TCP congestion control

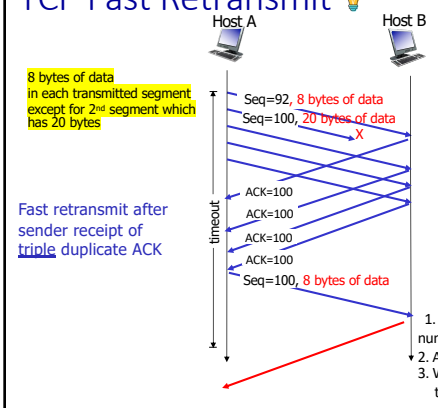


6

Warmup: TCP Fast Retransmit

3 duplicate ACKs received indicates 3 additional segments have arrived

What does it mean?
→likely segment is lost
→Retransmit!



12

Let's start thinking about congestion

Chapter 3 Section 6

Cost of congestion:

- Resources not fully utilized
- End systems receive poor performance

14

Retransmissions due to congestion

- Cause of packet loss during congestion
  - Buffer overflow in routers
  - Too many sources sending data at too high rate
- Treat symptoms of congestion
  - Various mechanisms to provide reliable data transfer
- Treat the cause of congestion?
- Is congestion "bad"?

15

Why worry about congestion? Book Section 3.6

Consequence of Congestion:

- Informally: "too many sources sending too much data too fast for *network* to handle"
- Consequences:
  - long delays (ask why)
  - packet loss (ask why)
  - Throughput vs. Delay as load increases

Congestion Control: too many senders, sending too fast

vs.

Flow Control: sender too fast for receiver

We will look at 3 congestion scenarios

16

Sending Rate vs.Throughput

- At the Sender: Transmission Rate
  - "Offered Load"
  - Original data + retransmissions
- At the Receiver: Throughput
  - Rate **original** data is delivered to Application Layer
    - Omit duplicates and retransmissions

17

3 scenarios

What happens as hosts increase their transmission rate and the network becomes congested?

18

Three Congestion Scenarios

What happens as hosts increase their transmission rate and the network becomes congested?

Scenario Assumptions:
Sending hosts share router on path from source to destination

Scenario 1: infinite buffer

Scenario 2: congestion, packet loss

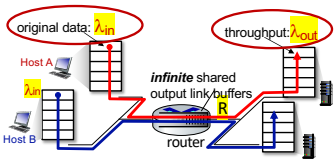
Scenario 3: congestion, multi-hop path

19

scenario 1 Causes/costs of congestion

Simplest scenario:

- Two flows
- App-layer input rate = λ_{in}
- One router, shared infinite buffer
  - no retransmissions needed
- Router output link R bits/sec
- Receiver throughput λ_{out}

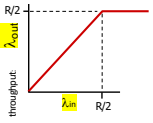


Throughput:

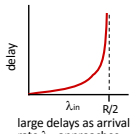
- λ_{in} < R/2
- What happens to the per connection throughput λ_{out} as arrival rate λ_{in} increases?
  - Upper bound on λ_{out}

Delay: (ouch!)

Queue grows without bound



maximum per-connection throughput: R/2



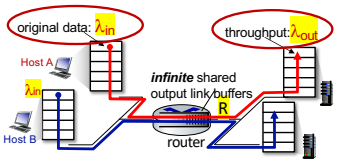
large delays as arrival rate λ_{in} approaches capacity

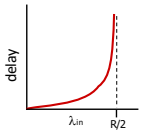
20

scenario 1 Causes/costs of congestion

Cost of congested network:

Large queueing delays as packet arrival rate nears link capacity





large delays as arrival rate λ_{in} approaches capacity

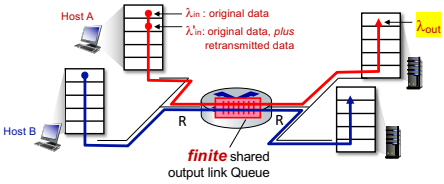
21

scenario 2 Causes/costs of congestion

- More realistic: ***finite*** buffer/queue at the router

Some perfect knowledge:

- Packets can be dropped at router – requiring retransmission
  - Sender knows exactly when packet has been dropped (perfect timer)
- Sender retransmits lost, timed-out packet
  - App-layer original input = λ_{in}. Appl-layer output (new data only): = λ_{out}
  - Transport-layer input includes *retransmissions* : λ'_{in}



Per connection throughput λ_{out} will be reduced because of the time to do retransmissions

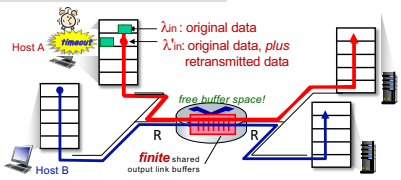
22

scenario 2 Causes/costs of congestion

Even more realistic -- *some un-needed duplicates created and transmitted due to imperfect timer:*

1) Packets can be dropped at router – requiring retransmissions

2) Sender can also time out prematurely, sending *two* copies, *both* of which are delivered (un-needed duplicates)



Per connection throughput λ_{out} will be reduced even more because of the un-needed duplicate transmissions

23

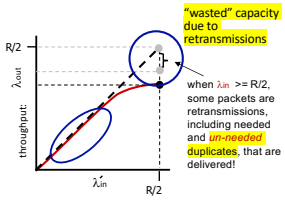
scenario 2 Causes/costs of congestion

Cost of congested network:

More work (retransmission) for receiver throughput

- Un-needed retransmissions: multiple copies of a packet carried over links and processed at router
  - decreasing maximum achievable throughput

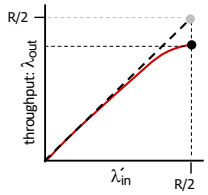
λ_{out} is reduced … < R/2



24

Homework: Study Scenarios 1 and 2 here and in book

- Understand the basic idea the issues
- Routers have finite buffers
- More work (retransmission) for given receiver throughput
- Throughput reduced < R/2 because of retransmissions and timeouts



26

Homework: Read Scenario 3 in book

- 2 hop network → what happens?
- Congestion collapse!

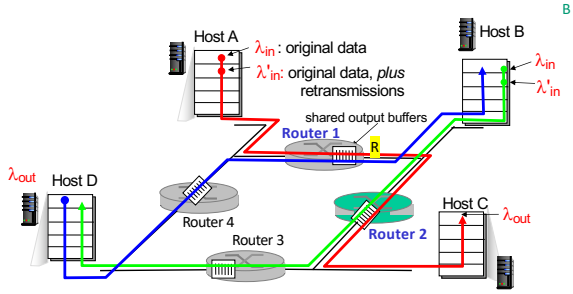
27

Scenario 3 Causes/costs of congestion: *

- Multi-hop paths
- Output capacity/rate of routers = R bits/sec
- Finite buffers
  - Timeout/Retransmissions

Router 1:
A → C
D → B

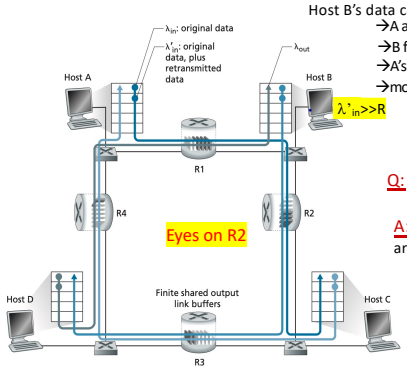
Router 2:
A → C
B → D



28

Causes/costs of congestion:

Host A: 1)data is rate-limited leaving R1 →
2) λ'_{in} --- already pkt loss at R1
Host B's data can arrive λ'_{in} >>R
→A and B compete for space in R2
→B fills immediately
→A's pkts find a full Q!
→most of A's traffic is retransmissions

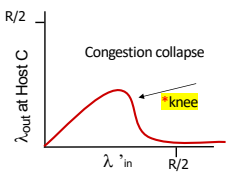


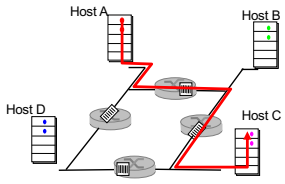
Q: what happens as λ_{in} and λ'_{in} increase ?

A: A's pkts arriving at R2 (red) are dropped, , λ_{out} at C → 0

30

Causes/costs of congestion





Scenario 3

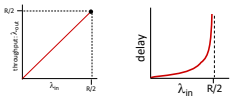
- Low λ_{in}: overflow rare and $\lambda_{in} = \lambda_{out}$
- As λ_{in} grows large: empty buffer capacity at R2 filled by Host B
  →Host A starves.
- A—C throughput λ_{out} → 0

Congestion Collapse → another "cost" of congestion
- Upstream nodes starved
- Transmission capacity upstream was wasted
- Throughput → 0

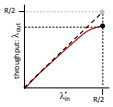
31

Causes/costs of congestion: insights

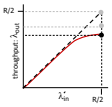
- Throughput can never exceed capacity
- Delay increases as capacity approached



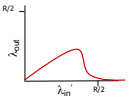
- Loss/retransmission decreases effective throughput



- Un-needed duplicates further decreases effective throughput



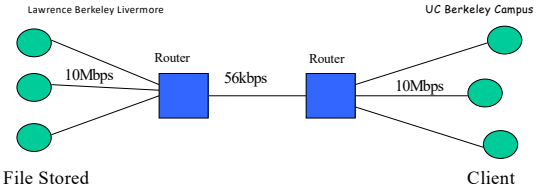
- Up-stream transmission capacity / buffering wasted for packets lost downstream



32

Homework: WATCH This Video

- One of designers of TCP Congestion Control
- *https://www.youtube.com/watch?v=QP4A6L7CEqA*
- Van Jacobson discusses the congestion collapse of the 1980s and how he figured out how to fix it – Slow Start!
- Similar article: https://www.wired.com/2012/05/van-jacobson/



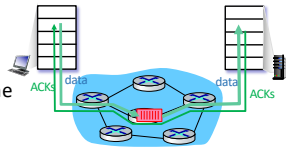
Watch the video! What problem is Van Jacobson describing?

33

Two Approaches towards congestion control

Network-assisted congestion control:

- Help from the network
- Router monitors state and provides direct feedback from the network



End-end congestion control

- no explicit feedback from network

34

Vint Cerf and Robert Kahn – Fathers of the Internet/TCP IP

- Inventors of TCP/IP in 1970s
  - Before laptops, mobile phones, tablets, workstations, Wifi, DSL, Web …. !
  - Saw the need to connect different devices and different yet-to-be defined link layers
    - Satellite, wireless, undersea cables
    - https://www.nytimes.com/interactive/2019/03/10/technology/internet-cables-oceans.html
    - 'People think that data is in the cloud, but it's not. It's in the ocean.' –Jayne Stowell, Google undersea cable
  - 2004 received Turing Award
    - Like Nobel Prize for computer networking
- Video: https://www.youtube.com/watch?v=lLiQnw0b-YQ (12 minutes)

35

What is congestion to TCP? A lost data segment

- Is packet loss the ONLY sign of congestion?
- Is packet loss always a sign of congestion?
- Narrow view on congestion

36

TCP Congestion Control

- What should Sender do?

37

How does Sender figure out packet loss?

1) The RTO timer expires → probably a packet loss
   - Packet loss = congestion
   - An ACK arrives → so far so good
2) 3 Duplicate ACKs arrive → probably a packet loss
   - However, sometimes the network is still delivering data
     - How do we know that?

38

TCP congestion control: transmission rate adjustment

Given a packet loss:

Consider

1. How to adjust rate?
2. What should the rate be?
3. What are the Sender's algorithms to change its rate?

39

Goals of TCP congestion control

- Goal: Optimal transmission rate given network conditions
  - Transmit at maximum rate, BUT
    - slow down when the network is congested
    - speed up when network is not congested
- Issues:
  - Only feedback is from the Receiver (ACKs) or timeouts at Sender
  - No feedback from network

Key to adjustment – remember, pipelined protocol!

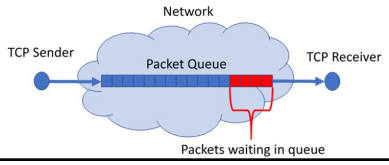
cwnd: "Congestion Window"

(*rwnd*: receiver's limiting window)

Sender Window = *min*(*cwnd*,*rwnd*)

(Usually *rwnd*>>*cwnd*, so we can focus on *cwnd*)

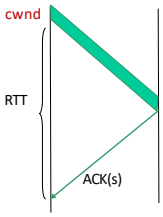
- *cwnd*
  - Dynamic variable, function of perceived congestion
  - Sender's view of ideal Window Size for current network conditions
  - number of unACKed bytes "in flight"



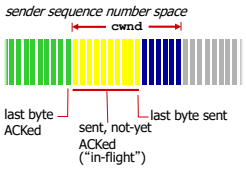
1. How can the Transmission Rate be changed?

- Sender can adjust transmission rate by changing *cwnd*
- Transmission rate:
  - every RTT, *cwnd* bytes transmitted
  - Wait for ACKS before sending more segments

$$\text{rate} \approx \frac{cwnd}{\text{RTT}} \text{ bytes/sec}$$



TCP Congestion Control: details



Sender limits window size:

$$\text{LastByteSent} - \text{LastByteAcked} \le \texttt{cwnd}$$

TCP sending rate:

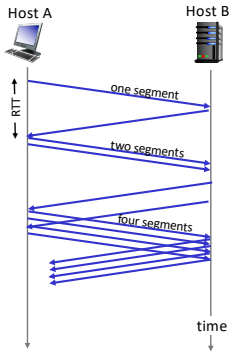
$$\text{rate} \approx \frac{\texttt{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

2. Two algorithms to adjust rate

- TCP switches between 2 modes:
  - Slow-start
  - Congestion avoidance
- And one optimization
  - Fast Recovery

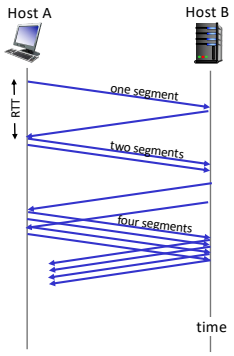
TCP Slow Start: exponential increase

- When used?
  - Connection begins
  - After timeout
- Initially, cwnd = 1 MSS
- Increase rate exponentially
  - cwnd doubles every RTT
  - Increment *cwnd* for every ACK received
- What is happening to the sender transmission rate?



TCP Slow Start: when to stop?

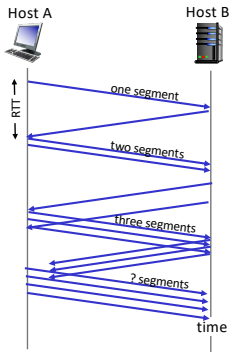
- Aggressive bandwidth probing
- When to stop?
  - Packet loss
  - `ssthresh` exceeded (initial threshhold = 64k) (look at graph)
- Summary: slow initial rate, but ramps up exponentially
- Probing!



46

TCP Congestion Avoidance: Linear Increase

- Steady state operation
- More conservative approach to growth of *cwnd* than Slow Start
  - `cwnd` **increases** by 1 MSS per RTT
- Less aggressive bandwidth probing (slower) – when to stop?



47

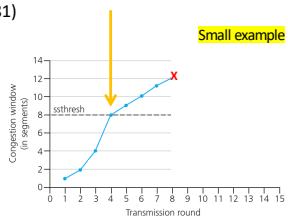
Slow Start → Congestion Avoidance

Q: How to set ssthresh?

- Initialization: ***ssthresh*** = 64K (RFC 5681)

Q: When should the exponential increase switch to linear?
→pivots when `cwnd > ssthresh`

Small example



Q: How to adjust `ssthresh` after Packet Loss?

- `ssthresh` is set to 1/2 of `cwnd` just before loss event
  - remember rate when loss occurred

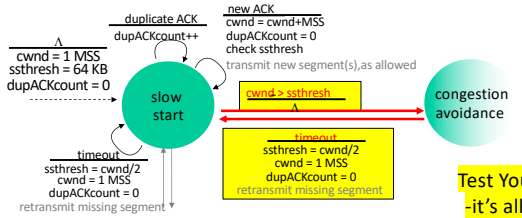
Q: What about packet loss? What to do with `cwnd`? (soon!)

48

Transitions into/out of Slow Start

Sender maintains **`ssthresh` and `cwnd`**

- When **`cwnd`** >= **`ssthresh`**: transition from Slow Start to Congestion Avoidance phase
- On loss event: reduce **`ssthresh`** to **`cwnd/2`**
  - (Allows remembering rate when congestion/loss occurred)



Test Yourself! -it's all here

Missing: Fast Recovery state and Fast Retransmit

49

How to adjust cwnd:

- What to do during Start Up, duplicate ACKs and timeout
  - Follow full state diagram

50

Reaction to packet loss depends on TCP version

- Many versions of TCP
- TCP Tahoe
  - any packet loss *cwnd* → 1
- TCP Reno
  - Loss recovered by Fast Retransmission: cwnd → 1/2
  - Timeout: *cwnd* → 1

51

Steady state and packet loss

(Steady State = Congestion Avoidance)

Additive Increase: →

Multiplicative decrease →
Reduce sending rate:

- Cut `cwnd` in half on loss detected by triple duplicate ACK (TCP Reno)
- Cut `cwnd` to 1 MSS when loss detected by timeout (TCP Tahoe and Reno)

- + ACKs: increase ***cwnd*** by 1 MSS per RTT
  - Linear increase
- - Loss:
  - ssthresh = ½ *cwnd*
  - If Timeout: *cwnd* →1
  - Else if Fast retransmit:
    - TCP Reno:
      cwnd = (*ssthresh*+3)
    - TCP Tahoe:
      cwnd = 1

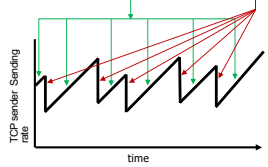
52

TCP congestion control: AIMD

- Steady State Approach: senders can increase sending rate until packet loss (congestion) occurs, then decrease sending rate on loss event

Additive Increase
increase sending rate by 1 MSS every RTT until loss detected

Multiplicative Decrease
cut sending rate in half at each loss event

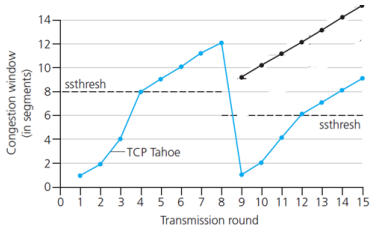


AIMD sawtooth behavior: *probing* for bandwidth

Assume losses determined by 3 duplicate ACKs/Reno

53

TCP Tahoe and TCP Reno



Identify parts of graph:

- slow start vs. congestion avoidance
- *cwnd*
- *ssthresh*
- Tahoe
- Reno

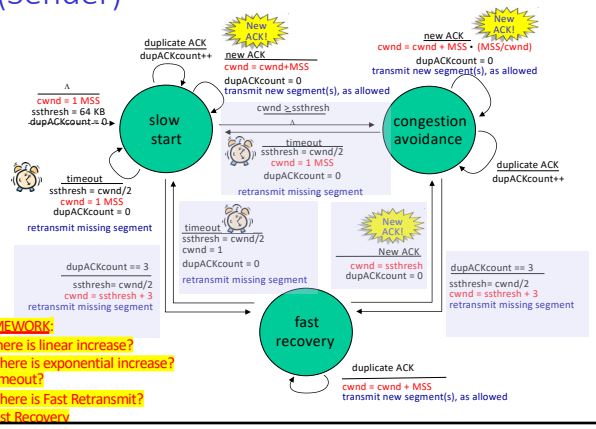
55

TCP: Fast Recovery (Reno version)

- Think about duplicate ACK
  - Receiving an ACK means something …. What does it mean?
- Fast Recovery allows:
  - Once 3 duplicate ACKs have been received:
    - For each additional duplicate ACK received, temporarily increase *cwnd* by 1 segment until the loss is recovered
    - Once recovered, reset `cwnd` and transition to Congestion Avoidance state
- See FSM for details (next)

56

Summary: TCP Congestion Control (Sender)



HOMEWORK:
1.Where is linear increase?
2. Where is exponential increase?
3. Timeout?
4. Where is Fast Retransmit?
5. Fast Recovery

57